



Análisis Exploratorio de Datos (EDA) de un CSV

El objetivo es convertir datos crudos en información estructurada y detectar problemas de calidad antes del análisis.

Usando un **Jupyter Notebook** (el método más común) y la librería **Pandas**, seguirías un flujo como este:

Paso 1: Cargar y Explorar los Datos

1. **Abre** Jupyter Notebook o JupyterLab desde el Anaconda Navigator.
2. **Importa Pandas** y lee tu archivo (CSV, Excel, etc.):

Python

```
import pandas as pd  
df = pd.read_csv('nombre_archivo.csv') #df: dataframe
```

3. Exploración inicial:

- Muestra las primeras filas: `df.head()`
- Obtén un resumen de la estructura de datos (tipos de datos y valores no nulos):
`df.info()`
- Obtén un resumen estadístico: `df.describe()`

Paso 2: Manejar Valores Faltantes (Missing Data)

1. **Identifica** la cantidad de valores faltantes por columna:

Python

```
df.isnull().sum()
```

2. Decide la estrategia:

- **Eliminar** filas/columnas con demasiados valores faltantes: `df.dropna(axis=0)` (elimina filas), `df.dropna(axis=1)` (elimina columnas).
- **Imputar** (rellenar) los valores faltantes con un valor (media, mediana, moda o un valor constante). Por ejemplo:



Python

```
# Rellenar con la media de la columna 'Edad'  
df['Edad'].fillna(df['Edad'].mean(), inplace=True)
```

Paso 3: Identificar y Eliminar Duplicados

1. **Revisa** si hay filas completamente duplicadas:

Python

```
df.duplicated().sum()
```

2. **Elimina** los duplicados:

Python

```
df.drop_duplicates(inplace=True)
```

Paso 4: Estandarizar y Corregir Formatos de Datos

1. **Corregir tipos de datos:** Por ejemplo, si una columna de fechas se cargó como texto, conviértela:

Python

```
df['fecha'] = pd.to_datetime(df['fecha'])
```

2. **Estandarizar texto:** Convertir a minúsculas, eliminar espacios extra, etc., para garantizar la consistencia (útil para campos categóricos):

Python

```
df['columna_texto'] = df['columna_texto'].str.lower().str.strip()
```

Conversión de Tipos: Si una columna se leyó como texto (`object`) pero debería ser numérica o de fecha, conviértela:

- **A Numérico:** `df['columna_num'] = pd.to_numeric(df['columna_num'], errors='coerce')`. El argumento `errors='coerce'` es útil, ya que cualquier valor que no sea un número se convertirá en `NaN`, permitiéndote tratarlo en el siguiente ciclo de limpieza.



- **A Fecha:** `df['columna_fecha'] = pd.to_datetime(df['columna_fecha'])`.

Filtrado Lógico y Manejo de Atípicos

Filtra el *DataFrame* para eliminar registros que, aunque técnicamente no son nulos o duplicados, son erróneos o irrelevantes para tu análisis.

1. **Filtrar por Lógica:** Elimina registros fuera de un rango válido. Por ejemplo, si una columna de edad tiene valores negativos o mayores a 100: `df = df[df['Edad'].between(18, 99)]`.
2. **Filtrar por Lista:** Conserva solo un conjunto específico de categorías: `df = df[df['Estado_Civil'].isin(['Soltero', 'Casado'])]`.